Foundations of Programming

Assignment No.: 20

PROBLEM STATEMENT: Write a C function to swap two numbers with and without using pointers.

OBJECTIVE:

- To understand the concept of function calls in the C programming language.
- To learn the difference between call by value and call by reference.
- To demonstrate swapping of two numbers with and without pointers.

THEORY:

In the C programming language, swapping two numbers can be performed in two ways: without pointers (call by value) and with pointers (call by reference).

When swapping numbers without pointers, the values are passed to the function by value. This means that a copy of the variables is sent to the function, and any changes made inside the function do not affect the original variables.

When swapping numbers with pointers, the addresses of the variables are passed to the function. This allows the function to directly modify the original variables in memory. Pointers therefore enable efficient data manipulation and are widely used in C for handling memory and complex data structures.

PLATFORM: Linux

ALGORITHM:

Without Using Pointers

1. Start
2. Input two numbers a and b
3. Create a temporary variable temp
4. Perform swapping:
   - temp = a
   - a = b
   - b = temp
5. Display swapped values
6. Stop

Using Pointers

1. Start
2. Input two numbers a and b
3. Pass addresses of a and b to function
4. In function:
   - Create pointer variables
   - Create temporary variable temp
   - temp = *a
   - *a = *b
   - *b = temp
5. Return to main function

6. Display swapped values
7. Stop

INPUT:

```
#include <stdio.h>
void swapwithoutpointers(int a, int b);
void swapwithpointers(int *p, int *q);
int main() {
    int a, b;
    printf("Enter two numbers (a and b): ");
    scanf("%d %d", &a, &b);
    printf("\n--- Without Pointers ---\n");
    swapwithoutpointers(a, b);
    printf("Result in main after swapWithoutPointers: a = %d, b = %d\n", a, b);
    printf("\n--- With Pointers ---\n");
    swapwithpointers(&a, &b);
    printf("Result in main after swapWithPointers: a = %d, b = %d\n", a, b);
    return 0;
}
void swapwithoutpointers(int a, int b) {
    int temp;
    temp = a;
    a = b;
    b = temp;
    printf("Inside function: a = %d, b = %d\n", a, b);
}
void swapwithpointers(int *p, int *q) {
    int temp;
    temp = *p;
    *p = *q;
    *q = temp;
    printf("Inside function: *p = %d, *q = %d\n", *p, *q);
}
```

```c
#include <stdio.h>

void swapwithoutpointers(int a, int b);
void swapwithpointers(int *p, int *q);

int main() {
    int a, b;

    printf("Enter two numbers (a and b): ");
    scanf("%d %d", &a, &b);

    printf("\n--- Without Pointers ---\n");
    swapwithoutpointers(a, b);

    printf("Result in main after swapWithoutPointers: a = %d, b = %d\n", a, b);

    printf("\n--- With Pointers ---\n");

    swapwithpointers(&a, &b);

    printf("Result in main after swapWithPointers: a = %d, b = %d\n", a, b);

    return 0;
}

void swapwithoutpointers(int a, int b) {
    int temp;
    temp = a;
    a = b;
    b = temp;
    printf("Inside function: a = %d, b = %d\n", a, b);
}

void swapwithpointers(int *p, int *q) {
    int temp;
    temp = *p;
    *p = *q;
    *q = temp;
    printf("Inside function: *p = %d, *q = %d\n", *p, *q);
}
```

OUTPUT:

Enter two numbers: 10 20

Before swapping: a = 10 b = 20

Inside function (without pointers): a = 20 b = 10

After swapping without pointers: a = 10 b = 20

After swapping with pointers: x = 20 y = 10

CONCLUSION: Thus, the C program successfully demonstrates the difference between swapping values using call by value and call by reference. Using pointers allows the function to modify the original variables directly.

FAQs:

1. What is the difference between call by value and call by reference in C?
2. How do pointers help modify original variables inside a function?
3. Why is swapping using pointers more effective in certain programs?